



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Практическая работа № 2

по дисциплине «Технологии разработки приложений на базе фреймворков»

Тема работы: Основы Python: коллекции, функции, циклы и расширенные возможности Python

Выполнил: студент группы ЭФБО-11-24
(группа)

Мишуков В.Р.
(подпись) (Фамилия И.О.)

Принял: доц.
(должность)

Ящун Т.В.
(подпись) (Фамилия И.О.)

Москва
2026

1. Индивидуальный проект

- Название проекта: Сервис регистрации продавцов на ярмарку (FairVendor), тема № 433.
- Перечень функций, реализованных на этапе ПР2:
 1. storage.py: `safe_load_json()`, `safe_save_json()`, `load_fairs()`, `save_fairs()`, `load_vendors()`, `save_vendors()`, `load_applications()`, `save_applications()` — надежное персистентное сохранение и считывание данных ярмарок, участников и заявок в формате JSON с гарантированным закрытием файлов через контекстный менеджер `with open` и перехватом исключений `FileNotFoundError`, `json.JSONDecodeError`;
 2. vendors.py: `validate_inn()` (проверка 10- или 12-значного числового формата ИНН), `add_vendor()` (регистрация продавца в коллекции со строгой валидацией полей и контролем уникальности ИНН), `get_vendor_by_id()` (быстрая выборка продавца по идентификатору), `find_vendor_by_name()` (поиск участников по регистронезависимой подстроке имени бренда), `filter_vendors_by_category()` (поэлементная фильтрация продавцов с использованием генератора и оператора `yield`), `sort_vendors()` (сортировка участников по наименованию или стажу с применением анонимных `lambda`-функций);
 3. applications.py: `get_occupied_space()` (расчет суммарной занятой площади на ярмарке по активным заявкам), `is_space_available()` (проверка наличия достаточной свободной площади на площадке), `create_application()` (создание и добавление заявки с проверкой лимита мест, валидацией документов и вычислением сбора), `cancel_application()` (отзыв и аннулирование брони с освобождением торгового пространства), `calculate_fair_statistics()` (вычисление сводной аналитики: процент занятости, выручка, число одобренных заявок), а также функции ПР1 (`validate_vendor_data`, `calculate_participation_fee`, `determine_application_status`, `generate_registration_card`);
 4. utils.py: `input_int()`, `input_float()`, `input_date()` (безопасный консольный ввод с циклической валидацией и перехватом `ValueError`), `format_currency()` (финансовое форматирование);
 5. main.py: консольный интерфейс с интерактивным меню на 10 пунктов и демонстрационным режимом (`--demo`).
- Структура модулей проекта: проект декомпозирован на слабосвязанные логические компоненты: `main.py` (точка входа, меню и диспетчеризация), `vendors.py` (управление участниками), `applications.py` (управление заявками и расчетом площадей), `storage.py` (работа с файловой подсистемой), `utils.py` (ввод-вывод и форматирование), каталог `data/` (файлы постоянного хранения), каталог `tests/` (набор тестов `test_vendors.py`, `test_applications.py`), `test_main.py` (обратная совместимость), `requirements.txt` и `.flake8`.
- Формат хранения данных: данные персистентно хранятся в формате JSON в каталоге `data/`:
 1. `fairs.json` — список словарей ярмарок (ключи: `id`, `name`, `location`, `date`, `base_rate`, `total_space`);
 2. `vendors.json` — реестр продавцов (ключи: `id`, `name`, `inn`, `category`, `has_documents`, `experience_years`);
 3. `applications.json` — журнал заявок (ключи: `id`, `vendor_id`, `fair_id`, `requested_space`, `fee`, `status`).
- Ссылка на Git-репозиторий: <https://github.com/aplaygs/IP.git>

2. Результат работы

В результате выполнения практической работы № 2 (ПР2):

- освоены и применены на практике встроенные коллекции Python: динамические списки (list) и словари (dict) для хранения структурированных сущностей ярмарок, продавцов и заявок, а также множества (set);
- исходный монолитный код первого этапа (ПР1) декомпозирован на логические модули по зонам ответственности: vendors.py, applications.py, storage.py, utils.py и main.py;
- реализован переход от временных переменных в оперативной памяти к постоянному хранению данных в файлах fairs.json, vendors.json, applications.json с автоматической инициализацией и валидацией данных;
- обеспечена надежная работа с файлами с использованием контекстного менеджера with open(), гарантирующего корректное освобождение дескрипторов при любых сценариях выполнения;
- реализована отказоустойчивая обработка исключений: перехват FileNotFoundError и json.JSONDecodeError при сбоях чтения файлов, ValueError при некорректном пользовательском вводе и бизнес-валидации (дубликат ИНН, дефицит площади стенда);
- применены расширенные возможности языка: генераторы с оператором yield для эффективной фильтрации участников по товарным категориям и анонимные lambda-функции для гибкой сортировки;
- все ключевые функции снабжены стандартными аннотациями типов аргументов и возвращаемых значений, а также подробными docstring-комментариями;
- исходный код проверен линтером flake8: устранены замечания PEP 8 (длина строк, неиспользуемые импорты), получен чистый вывод;
- разработана система модульного тестирования на базе фреймворка pytest в каталоге tests/ (20 тестов, 100% passed);
- обновлена проектная документация README.md, зафиксирован единый итоговый коммит «Практика 2» и выполнен push в удаленный репозиторий GitHub.

3. Вывод

В ходе выполнения практической работы № 2 были освоены и закреплены ключевые конструкции языка Python, необходимые для разработки надежных прикладных систем: структуры данных (списки, словари), модульная декомпозиция программного кода, контекстные менеджеры для безопасного ввода-вывода, генераторы и лямбда-выражения. Реализована обработка исключительных ситуаций, исключающая аварийное завершение программы при ошибочных действиях пользователя или сбоях файловой системы. Индивидуальный проект «Сервис регистрации продавцов на ярмарку» развит до полнофункционального модульного консольного приложения с персистентным JSON-хранилищем, агрегацией аналитики и автоматизированным тестовым покрытием pytest. Код полностью соответствует стандарту PEP 8 и подготовлен к реализации объектно-ориентированной модели (ООП) на этапе ПР3.

Москва
2026